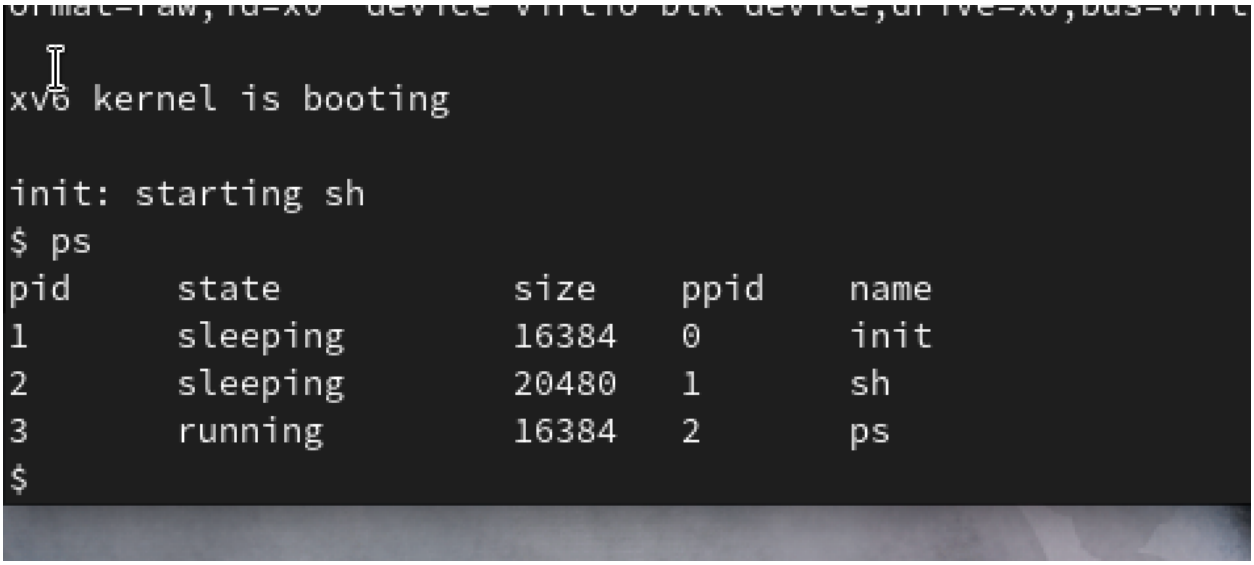
CS4475 Fall 2026 Lab Report
Frida Peña Submitted on October 14, 2026
fmpena@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

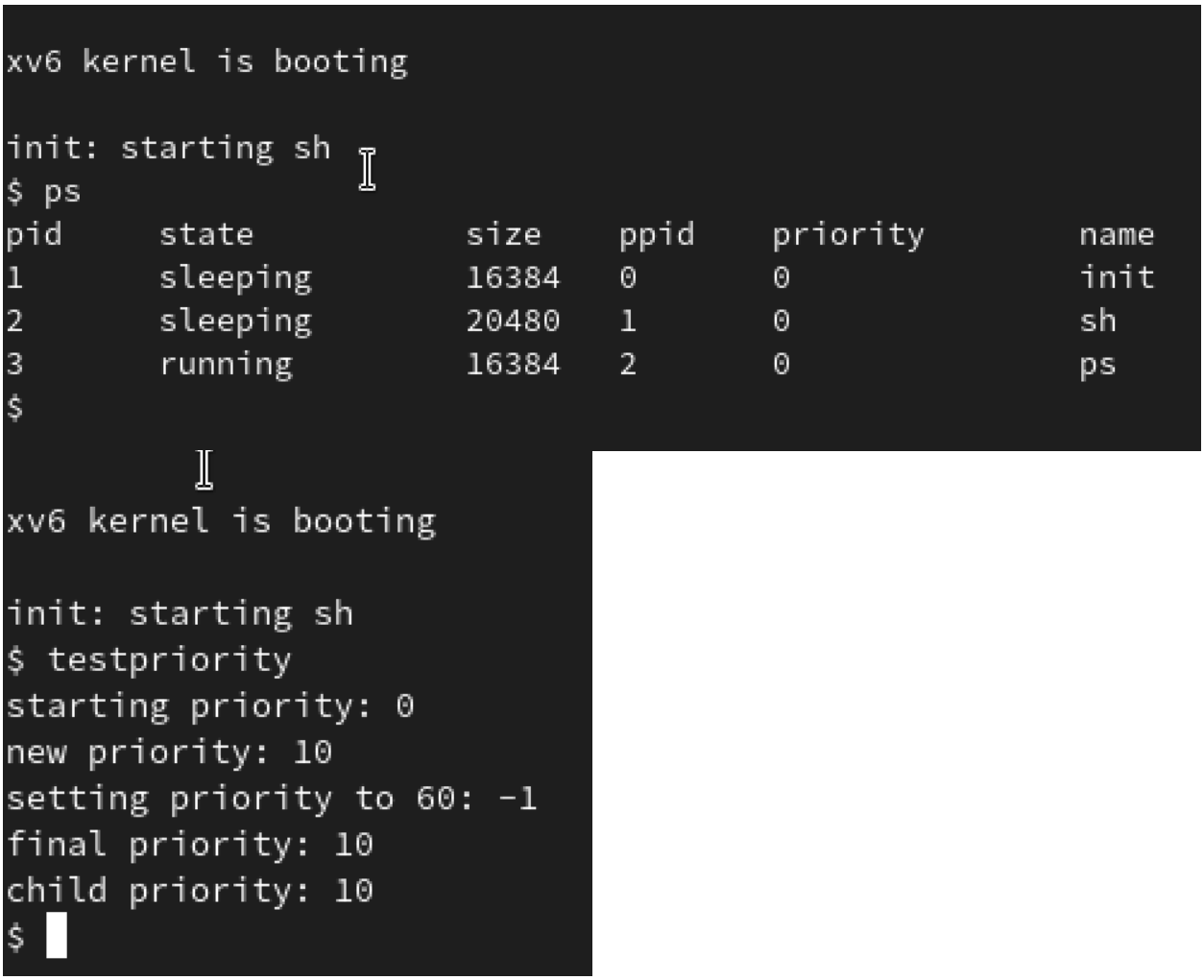
For this task, I implemented the getprocs() system call so that I could get information about the processes running in xv6. I started by adding the pstat structure to store information such as the process ID, state, memory size, parent ID, and name. I also had to update the system call files so that the ps command would be able to access this information. In proc.c, I added a function that goes through the processes, skips the ones that aren't being used, and collects their information. I also used locks when accessing the process information and copyout() to send the results back to the user program.The files I had to modify were kernel/pstat.h, kernel/proc.c, kernel/defs.h, kernel/sysproc.c, kernel/syscall.c, kernel/syscall.h, user/user.h, user/usys.pl, user/ps.c, and the Makefile. To test my implementation, I compiled xv6 and ran the ps command. The output showed three processes: init, sh, and ps. Each process displayed its ID, state, size, parent ID, and name. The results matched what I expected, so I was able to confirm that getprocs() was working correctly.



Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

For this task, I added a priority field to each process so that xv6 could keep track of its priority. I initialized the priority to 0 and updated kfork() so that child processes would inherit their parent's priority. I also implemented getpriority() and setpriority() to allow processes to check and change their priorities. I updated getprocs() to collect the priority information and modified ps.c to display it along with the other process information. To test my implementation, I first

ran ps and confirmed that all processes started with priority 0. I also created a test program called testpriority.c, which successfully changed the priority from 0 to 10 and rejected an invalid priority of 60. I then tested priority inheritance and confirmed that the child process inherited its parent's priority of 10.



Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

For this task, I added a readytime field to struct proc to keep track of when a process becomes runnable. I initialized the field to 0 and updated the functions that change a process's state to RUNNABLE, including userinit(), kfork(), yield(), wakeup(), and kkill(). I also updated getprocs() to calculate the process's age by subtracting readytime from the current clock ticks. This calculation is only done for processes in the RUNNABLE state. I then modified pstat.h and ps.c to include an age column. To test my implementation, I compiled xv6 and ran the ps command to make sure the new column appeared. I also created a test program called testage.c that runs five child processes competing for one CPU. The test reported a maximum waiting age of 4 ticks, which showed that the age calculation was working. The results confirmed that xv6 could track how long a runnable process had been waiting for CPU time.

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    priority        age     name
1       sleeping        16384   0       0               0       init
2       sleeping        20480   1       0               0       sh
3       running         16384   2       0               0       ps
$
```

```
xv6 kernel is booting

init: starting sh
$ testage
maximum child age: 4 ticks
$
```

Task 4. (15 pts) Implement a priority-based scheduler.

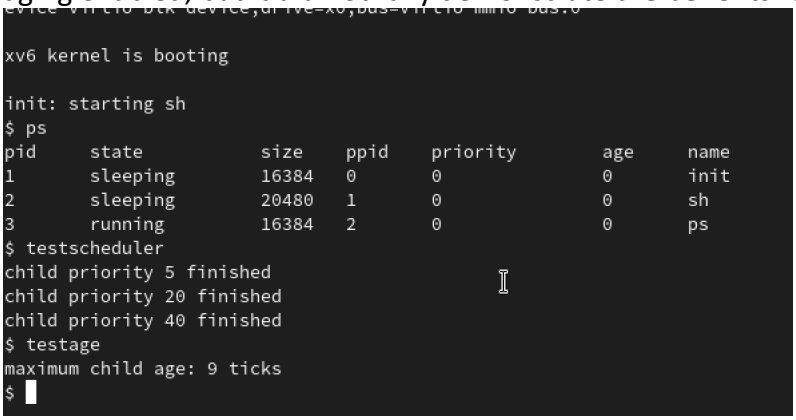
For this task, I modified the scheduler in proc.c to support priority-based scheduling. I added a PRIORITY_SCHED option in param.h so that I could switch between the original round-robin scheduler and the new priority scheduler. When priority scheduling is enabled, the scheduler goes through the process table and looks for the runnable process with the highest priority. Once it finds that process, it selects it to run. If priority scheduling is disabled, xv6 uses the original round-robin scheduler instead. To test my changes, I compiled xv6 and ran the ps command to make sure the system was still working. I also ran testpriority.c to confirm that the priority system calls were still working. After that, I created testscheduler.c, which runs three child processes with priorities 5, 20, and 40. All three processes completed successfully, and xv6 returned to the shell without errors. However, since the processes were created one at a time, this test did not fully confirm that higher-priority processes always run first.

```
xv6 kernel is booting

init: starting sh
$ testscheduler
child priority 5 finished
child priority 20 finished
child priority 40 finished
$
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

For this task, I decided to use a tiered aging policy to help prevent lower-priority processes from waiting too long for CPU time. Instead of increasing the priority by the same amount every tick, I divided the waiting time into three levels. Processes that wait less than 10 ticks do not receive a boost, processes that wait between 10 and 29 ticks receive a boost of 5, and processes that wait 30 ticks or longer receive a boost of 15. The effective priority is also limited to 99. I implemented this in the scheduler by calculating how long each runnable process had been waiting and then adding the appropriate boost to its base priority. This does not permanently change the process's original priority. To test my changes, I compiled xv6 and ran testscheduler and testage. Both programs ran successfully, and testage reported a maximum waiting age of 9 ticks. I also created timingtest.c to compare response time and turnaround time with aging enabled and disabled. With aging enabled, the average response time was approximately 0.67 ticks and the average turnaround time was 1.33 ticks. Without aging, the averages were approximately 0.33 ticks and 0.67 ticks. These results showed that aging did not improve the average times in my tests. One reason is that the processes finished quickly and did not wait long enough to reach the aging thresholds. One difficulty I ran into was testing whether aging made a difference, since the results were very small and could change between runs. I learned that aging is mainly intended to improve fairness for processes that wait longer, rather than always reducing execution times. My current test confirmed that the scheduler could run with aging enabled, but it did not fully demonstrate the benefits for longer-waiting processes.



```
qemu-system-riscv64 -machine virt -bios none -kernel kernel
evice virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ timingtest
priority 40: response 0, turnaround 1
priority 20: response 1, turnaround 1
priority 5: response 1, turnaround 2
average response: 0 ticks
average turnaround: 1 ticks
$
```

Top results: Aging enabled. ^

Bottom results: Aging disabled.

```
xv6 kernel is booting

init: starting sh
$ timingtest
priority 40: response 0, turnaround 0
priority 20: response 0, turnaround 1
priority 5: response 1, turnaround 1
average response: 0 ticks
average turnaround: 0 ticks
$
```

Task 6. (5 pts) Summary

For this assignment, I learned more about how process scheduling works in xv6 and how different scheduling policies can affect the order in which processes run. I started by implementing getprocs() and modifying the ps command to display process information. I then added process priorities, system calls to change them, and a way to track how long processes wait before running. One of the biggest things I learned was how priority scheduling and aging work together. Priority scheduling allows processes with higher priorities to be selected first, while aging is meant to help lower-priority processes that have been waiting for a longer time. I ran into several difficulties throughout the assignment, especially when modifying the scheduler and testing process age. At one point, I accidentally left part of the original scheduler

in proc.c, which caused compilation errors. I fixed this by reviewing the code and removing the duplicate section. I also had trouble testing process age because my first test kept returning 0. After changing the test to use multiple child processes, I was able to observe a nonzero waiting age. Overall, this assignment helped me better understand how xv6 manages processes and how small changes to the scheduler can affect the way the operating system runs.

Extra Credit Task (10 points).

For the extra credit task, I added getpriority and setpriority as built-in commands in the xv6 shell. I had already implemented the system calls in Task 2, so I modified sh.c to recognize the commands and execute them directly in the shell instead of creating a separate process. The getpriority command displays the shell's current priority, while setpriority allows the user to change it. Since these commands run inside the shell, the priority change stays in effect for future commands. This can help improve response time for interactive jobs by allowing the shell and the processes it creates to use a higher priority. To test my implementation, I first ran getpriority, which returned 0. I then used setpriority 10 and ran getpriority again, which returned 10. Finally, I ran ps and confirmed that both sh and ps had priority 10. This showed that the shell's priority changed successfully and that the child process inherited the new priority.

